

REVISÃO: LAÇOS DE REPETIÇÃO

Prof^a Me. Monique Emídio de Oliveira

O QUE SÃO LAÇOS DE REPETIÇÃO?

- Laços de repetição são estruturas que permitem executar um bloco de código várias vezes, enquanto uma condição for satisfeita.
- **Por que usar?**
- Sem repetição, teríamos que escrever o mesmo comando várias vezes.
- **Exemplo do cotidiano**
- Contar de 1 até 10
- Ler várias notas
- Repetir um menu até o usuário escolher sair

TIPOS DE LAÇOS EM C

Em C, existem 3 principais estruturas de repetição:

- while
- do...while
- for

Cada uma pode ser usada em situações diferentes.

ESTRUTURA while

- O while executa o bloco enquanto a condição for verdadeira.

Sintaxe

```
while (condicao) {  
    // comandos  
}
```

Importante

- A condição é testada antes da execução.
- Se a condição for falsa logo no início, o bloco não executa nenhuma vez.

EXEMPLO 1 COM while

```
#include <stdio.h>
```

```
int main() {  
    int i = 1;
```

```
    while (i <= 5) {  
        printf("%d\n", i);  
        i++;  
    }
```

```
    return 0;  
}
```

CUIDADO COM LOOP INFINITO

- Se a variável de controle não for atualizada, o laço pode nunca terminar.

Exemplo incorreto

```
int i = 1;

while (i <= 5) {
    printf("%d\n", i);
}
```

Problema

- i nunca muda
- A condição sempre será verdadeira
- O programa entra em **loop infinito**

ESTRUTURA do...while

- O do...while executa o bloco pelo menos uma vez, pois a condição é testada depois.

Sintaxe

```
do {  
    // comandos  
} while (condicao);
```

EXEMPLO 2 COM do...while

```
#include <stdio.h>

int main() {
    int i = 1;

    do {
        printf("%d\n", i);
        i++;
    } while (i <= 5);

    return 0;
}
```

Diferença principal

- Mesmo que a condição seja falsa no início, o bloco executa uma vez.

COMPARANDO while E do...while

while

- Testa antes
- Pode executar 0 vezes

do...while

- Testa depois
- Executa pelo menos 1 vez

Quando usar do...while?

- Menus
- Validação de entrada
- Quando a ação precisa acontecer antes da verificação

ESTRUTURA for

- O for é muito usado quando sabemos quantas vezes queremos repetir.

Sintaxe

```
for (inicializacao; condicao; incremento) {  
    // comandos  
}
```

Partes

- Inicialização: executada uma vez
- Condição: verificada a cada repetição
- Incremento: executado ao final de cada volta

EXEMPLO 3 COM for

- **Exibir números de 1 a 5**

```
#include <stdio.h>
```

```
int main() {  
    int i;  
  
    for (i = 1; i <= 5; i++) {  
        printf("%d\n", i);  
    }  
  
    return 0;  
}
```

CONTADORES

- Um contador é uma variável usada para contar quantas vezes algo aconteceu.

Exemplo

```
int contador = 0;  
contador++;
```

Uso comum

- Contar quantos números foram digitados
- Controlar repetições
- Contar valores positivos, negativos, pares etc.

CONTADORES

```
#include <stdio.h>
```

```
int main() {  
    int contador;
```

```
    for (contador = 1; contador <= 10; contador++) {  
        printf("Contador: %d\n", contador);  
    }
```

```
    return 0;  
}
```

ACUMULADORES

Um acumulador é uma variável usada para somar valores ao longo da repetição.

Exemplo

```
int soma = 0;  
soma = soma + valor;
```

Uso comum

- Somar números
- Calcular médias
- Totalizar compras

EXEMPLO 4: SOMA DE 1 A 10

```
#include <stdio.h>
```

```
int main() {  
    int i, soma = 0;  
  
    for (i = 1; i <= 10; i++) {  
        soma = soma + i;  
    }  
  
    printf("Soma = %d\n", soma);  
  
    return 0;  
}
```

BOAS PRÁTICAS

- Inicialize corretamente as variáveis
- Garanta que a condição de parada seja possível
- **Atualize a variável de controle** e evite loops infinitos acidentais
- Use nomes de variáveis claros
- Idente o código corretamente

Lista de exercícios no moodle